LI-ESKG: Latent Identities in Event-State Knowledge Graphs

A. CN
Independent Researcher
acn@gravitalia.com

Abstract

Modern systems rely on heterogeneous observations from cameras, microphones, textual documents, geolocation sensors, and machine-learning pipelines to construct evolving representations of the physical world. Event-State Knowledge Graphs (ESKGs) represent events and state transitions, but commonly assume that entity references are resolved before insertion. This assumption rarely holds in perception-driven systems. We define the Latent Identity Event-State Knowledge Graph (LI-ESKG), a formal integration architecture that separates immutable observations, versioned identity hypotheses, probabilistic inference records, decisions, and event-state histories. A transient inference layer generates candidates, compiles a factor graph, estimates an assignment distribution, and applies an explicit decision policy. A durable resolution ledger records the evidence, alternatives, model versions, decisions, dependencies, and revisions, while accepted relations are materialized in an existing authoritative knowledge graph. Assignments may be deferred. Merges are non-destructive. Splits and reassignments preserve earlier versions. We establish host non-interference before promotion, invariant preservation under valid commands, replay, and resource bounds, and we state the exact condition for local inference without claiming it as a new inference result. We then specify a Rust reference implementation using typed stable identifiers, an opaque provider interface, hot–cold data separation, bounded work queues, and batch persistence. LI-ESKG does not introduce a new probabilistic linkage algorithm. Its contribution is a precise and testable contract for integrating existing entity-resolution methods with persistent event-state graphs.

Keywords

entity resolution, latent identity, eskg, factor graph, uncertainty, streaming systems

1 Introduction

Knowledge graphs represent entities and relations in a form that supports integration, querying, and reasoning [1]. Temporal and event-centered extensions record how represented entities change. In the ESKG profile of Zang et al., states may trigger events, events may lead to states, states may evolve directly, events may contain other events, and events may occur at or influence physical nodes [18].

This representation usually assumes a resolved referent. This assumption rarely holds in perception-driven systems. Information systems do not observe entities. They observe heterogeneous outputs produced by sensors and machine-learning models. A camera produces a detection and an embedding. A microphone produces an acoustic segment and a speaker embedding. An OCR engine extracts a textual mention. A positioning system estimates a location and its uncertainty. Each observation is partial, uncertain, and modality-specific. Identities are not directly observed. They must be inferred from evidence acquired over time.

The difficulty is partly algorithmic. It is also ontological. Observations and identities denote different concepts and should not occupy the same level of abstraction. Observations constitute the primary evidence available to the system. Identities are explanatory hypotheses supported by that evidence. The integration architecture must therefore separate evidence, probabilistic inference, operational commitment, and the authoritative event-state graph into which accepted relations are eventually materialized.

Classical record linkage already formulates identity as an uncertain matching problem [4, 5]. Bayesian entity-resolution models infer latent assignments or partitions [10, 11]. Recent neural systems such as CrossER address generalized entity resolution across heterogeneous representations [13]; streaming or incremental linkage remains a separate implementation concern. Uncertain knowledge-graph construction reasons over extracted entities and facts [1, 2]. Multi-target tracking addresses uncertain measurement origin [12]. LI-ESKG adopts these results. It does not replace them.

The contribution is an integration contract. It has six parts.

(1) A three-plane model separates the authoritative host graph, a durable resolution ledger, and the active latent-identity workspace.
(2) A transient belief layer performs candidate retrieval and exact or approximate inference. The ledger remains the audit record, while accepted relations are materialized in the host graph.
(3) A Bayes-risk decision layer distinguishes assignment, new identity, noise, and abstention.
(4) Reassignment, merge, split, promotion, and reversal are append-only operations. They preserve prior ledger and host-graph versions.
(5) The formal analysis states conditional guarantees. It does not conflate graph projection, identity correctness, and causality.
(6) A Rust reference design exposes memory layout, concurrency, persistence, and measurement choices that determine performance.

This is a formal architecture paper. It reports no experiment. It therefore makes no empirical claim about accuracy, calibration, latency, throughput, or memory. Section 11 defines the evidence required for such claims.

2 Related Work and Positioning

2.1 Entity resolution

Fellegi and Sunter established a probabilistic decision framework for record linkage [5]. Later work introduced collective resolution, relational evidence, and scalable blocking [3, 4]. Bayesian formulations represent identity assignments as latent variables and retain posterior uncertainty [10, 11]. Recent neural systems such as CrossER address generalized entity resolution across heterogeneous representations [13]; they do not define the persistence and revision semantics studied here.

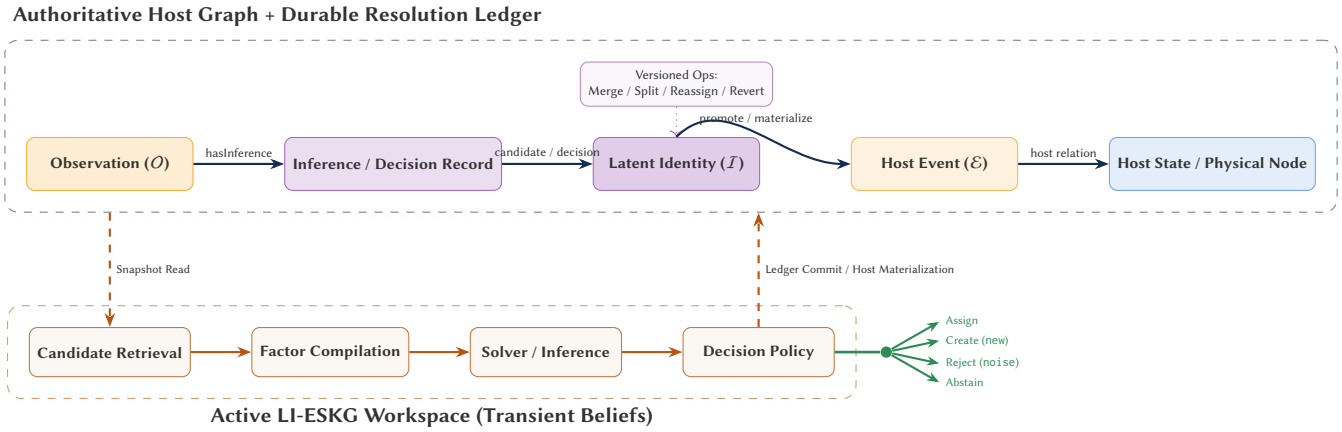

Figure 1: The authoritative host graph and durable resolution ledger are separated from the active LI-ESKG inference workspace.

The observation–identity distinction is therefore established. LI-ESKG is potentially novel as a formalization and integration architecture, but not as a probabilistic method or an entity-resolution algorithm. Its narrower claim is that uncertain linkage decisions can be made first-class, versioned objects beside an existing event-state system. Candidate generation, blocking, feature engineering, profile construction, and calibrated likelihood production remain external provider responsibilities.

2.2 Uncertain knowledge graphs

Knowledge-graph construction often starts from uncertain extractions. Knowledge Graph Identification jointly reasons about candidate entities, relations, and constraints [2]. Surveys cover uncertainty representation, fusion, conflict, and alignment [1]. LI-ESKG does not claim that knowledge graphs cannot represent uncertainty. It specifies the persistence and revision semantics required when uncertain references participate in an event-state history.

2.3 Tracking and factor graphs

Probabilistic data association computes measurement-origin probabilities instead of treating a nearest neighbor as certain [12]. Factor graphs represent factored distributions and support local message passing [8]. They are widely used in state estimation [7]. Sum–Product is exact on trees. Ordinary loopy Belief Propagation is approximate and need not converge [17]. The present architecture records this distinction and remains solver-independent.

2.4 Event-state graphs

An ESKG explicitly relates events, states, physical entities, and semantic knowledge. The profile used by Zang et al. defines six event-state relation roles: *Triggers*, *Leads to*, *Evolution*, *Contain*, *Occur*, and *Influence* [18]. LI-ESKG treats that schema as its motivating host profile. It does not rename those relations and does not treat them as an already standardized RDF vocabulary. Event-state edges encode the operational, temporal, conditional, or causal semantics declared by the host schema; LI-ESKG does not strengthen those claims.

2.5 Provenance and Semantic Web interoperability

PROV-O supplies a standard vocabulary for entities, activities, agents, usage, generation, derivation, revision, and invalidation [15]. RDF 1.2 triple terms and reifiers provide a representation mechanism for discussing a candidate or disputed proposition without necessarily asserting it as a current fact [14]. Nanopublications package an assertion with assertion provenance and publication information [6, 9]; SHACL provides machine-checkable RDF conformance constraints [16]. LI-ESKG does not require its numerical workspace or internal ledger to be stored as RDF. Instead, it defines a lossless interoperability projection from its durable records and host materializations to these standards.

3 Preliminaries and Scope

3.1 Base event-state graph

Let $\mathbb{T}$ be a totally ordered event-time domain. Following the motivating profile of Zang et al., a base ESKG at time t is a time-aware directed labeled graph

$$G_t^{\mathrm{ES}} = (V_t^{\mathrm{ES}}, E_t^{\mathrm{ES}}, \mathcal{R}^{\mathrm{ES}}, W_t),$$

where

$$V_t^{\mathrm{ES}} = \mathcal{S}_t \sqcup \mathcal{E}_t \sqcup \mathcal{P}_t \sqcup \mathcal{N}_t$$

contains state nodes, event nodes, physical-entity nodes, and semantic-knowledge nodes. The edge set satisfies

$$E_t^{\mathrm{ES}} \subseteq V_t^{\mathrm{ES}} \times \mathcal{R}^{\mathrm{ES}} \times V_t^{\mathrm{ES}},$$

and W_t contains optional confidence or weight information. The six motivating relation roles are

$$\begin{aligned} \mathrm{Triggers} &\subseteq \mathcal{S} \times \mathcal{E}, & \mathrm{LeadsTo} &\subseteq \mathcal{E} \times \mathcal{S}, \\ \mathrm{Evolution} &\subseteq \mathcal{S} \times \mathcal{S}, & \mathrm{Contain} &\subseteq \mathcal{E} \times \mathcal{E}, \\ \mathrm{Occur} &\subseteq \mathcal{E} \times \mathcal{P}, & \mathrm{Influence} &\subseteq \mathcal{E} \times \mathcal{P}. \end{aligned}$$

The central transition pattern is therefore

$$s_t \xrightarrow{\mathrm{Triggers}} e_t \xrightarrow{\mathrm{LeadsTo}} s_{t+1},$$

Table 1: Scoped positioning of LI-ESKG. Entries summarize common formulations, not universal limits.

Area	Primary object	Uncertainty	Online revision	Persistent event-state history
Record linkage	records and links	match or partition	batch; streaming variants exist	outside the core model
Bayesian entity resolution	observations and latent entities	posterior assignments or partitions	model-dependent	outside the core model
Uncertain KG construction	extracted entities and facts	confidence over nodes, links, or alignments	batch or incremental	possible, but not specific to ESKG
Multi-target tracking	measurements and tracks	measurement origin and target state	central concern	usually a track history
ESKG	events, states, physical nodes, and semantic knowledge	optional confidence or weight	explicit graph update	central concern
LI-ESKG	evidence, decisions, latent hypotheses, and host targets	versioned posterior and decision status	explicit and reversible	materialized into the host graph

while Evolution represents a direct state transition. This notation follows the cited ESKG profile. It does not imply that an official, universal RDF namespace for ESKG already exists, and it does not by itself establish an interventionist causal effect.

3.2 Two clocks and one version order

The system distinguishes event time t_e, ingestion time t_s, and commit version $v \in \mathbb{N}$. Event time describes the observed world. Ingestion time describes arrival. Commit version totally orders durable revisions. This distinction is required for late observations and retrospective correction.

3.3 Scope assumptions

Assumption 1 (Atomic reference). *Each observation submitted to identity inference refers to at most one physical entity. A frame containing three detected persons is decomposed into three observations before inference.*

Assumption 2 (Finite gated domain). *For each query observation and graph version, candidate generation returns a finite set. The complete latent universe may be unbounded.*

Assumption 3 (Deterministic replay). *Persistent transitions are deterministic given their inputs, component versions, policy version, and ordered transaction log. Randomized procedures persist their seed or their realized output.*

4 LI-ESKG Model

4.1 Host graph, durable ledger, and active workspace

Definition 1 (LI-ESKG system state). *At commit version v, an LI-ESKG deployment is the triple*

$$\Xi_v = (\mathcal{K}_v, \mathcal{L}_v, \mathcal{W}_v).$$

The authoritative host graph $\mathcal{K}_v$ contains accepted domain entities and relations, including the event-state subgraph. The durable resolution ledger $\mathcal{L}_v$ contains immutable evidence references, inference records, decision records, dependency links, revisions, and materialization receipts. The active workspace $\mathcal{W}_v$ contains candidate sets, latent identity hypotheses, provider-owned summaries, compiled factors, and solver state required for current inference.

The three components have different lifecycles. A hypothesis may disappear from $\mathcal{W}_v$ after resolution, promotion, retirement, or eviction. Its evidence and decision history remain reconstructible from $\mathcal{L}_v$. Accepted domain facts are materialized in $\mathcal{K}_v$. The ledger may be stored inside the same database as the host graph or in a separate append-only store; the contract does not require either physical arrangement.

The durable and host objects have disjoint semantics.

Observation $o \in \mathcal{O}$.
An immutable evidence record. It contains source, modality, timestamps, a payload reference, quality metadata, and a content hash. A correction creates another observation and a supersession relation.

Inference record $r \in \mathcal{F}$.
A durable inference result. It contains the gated domain, typed score contributions, a normalized distribution, provider, model, calibration, candidate, and solver versions, and diagnostics.

Decision record $d \in \mathcal{D}$.
A durable policy result. It references an inference record, the policy and loss versions, the selected action, and the validity interval. It is distinct from the inference record.

Identity hypothesis $i \in \mathcal{I}$.
An active explanatory hypothesis. It is not an assertion of metaphysical identity or a ground-truth entity. A historical descriptor remains in the ledger after the hypothesis leaves the active workspace.

Event, state, physical, and semantic nodes.
These belong to the authoritative host ESKG and retain the semantics and relation names of the host schema.

The logical relation signature is partitioned into

$$\mathcal{R}_v = \mathcal{R}_v^{\mathrm{HOST}} \sqcup \mathcal{R}_v^{\mathrm{LI}} \sqcup \mathcal{R}_v^{\mathrm{PROV}}.$$

The first subset belongs to the host schema. The second contains identity-resolution and transaction relations. The third is an interoperability projection aligned with PROV-O; it need not be the internal storage representation.

4.2 Observation contract

An observation is the tuple

$$o = (u, \sigma, m, t_e, t_s, \rho, q, h),$$

where u is a stable identifier, σ is the source, m is the modality, ρ references the payload, q contains quality and uncertainty metadata, and h is a content hash. Quality is not reduced to one generic confidence scalar. A position may expose covariance. A detector may expose class probability and localization quality. A text extractor may expose mention boundaries and a calibrated score.

4.3 Identity hypotheses and lifecycle

Each latent identity has status in

$$\{\texttt{active}, \texttt{dormant}, \texttt{resolved}, \texttt{merged}, \texttt{retired}\}.$$

An active identity may receive observations. A dormant identity leaves the hot belief set but remains retrievable. A resolved identity has been promoted to, or mapped onto, an authoritative host entity and therefore leaves the active workspace. A merged identity remains addressable through historical ledger versions. A retired identity is excluded from ordinary candidate retrieval.

Merges are versioned decisions. They never delete identity records or move historical edges. At version v, active merge decisions induce an equivalence relation $\sim_v$ by reflexive, symmetric, and transitive closure. A deterministic canonicalization function

$$c_v : \mathcal{I}_v \to \mathcal{I}_v$$

returns one representative per active equivalence class. A split is not merely the revocation of one merge edge. It supplies the intended partition, closes the relevant merge decisions, reallocates or marks ambiguous the supporting observations, rebuilds affected summaries, and invalidates dependent materializations through the dependency graph.

This mechanism guarantees internal referential consistency. It does not guarantee that one physical entity corresponds to exactly one identity hypothesis. That property is empirical and requires ground truth.

4.4 Association domain, inference records, and decisions

Candidate generation returns

$$C_v(o) \subseteq \mathcal{I}_v^{\text{latent}} \cup \mathcal{I}_v^{\text{known}},$$

where a known candidate is an authoritative host-entity reference and a latent candidate is an active identity hypothesis. The inference domain is

$$D_v(o) = C_v(o) \cup \{\texttt{new}, \texttt{noise}\}.$$

The label `new` denotes evidence for an unrepresented identity. The label `noise` denotes invalid, irrelevant, or non-entity evidence. An unresolved case is neither. It yields the action `abstain`.

Definition 2 (Inference record). *An inference record is*

$$r = (o, D, q, \theta, \eta, \gamma, [v_b, v_e)),$$

where q is a normalized distribution on D, θ identifies providers, statistical models, and calibration artifacts, η identifies candidate and solver configurations, γ contains diagnostics, and $[v_b, v_e)$ is the validity interval. If alternatives are truncated, the record stores the residual mass and truncation rule.

Definition 3 (Decision record). *A decision record is*

$$d = (r, \delta, \pi, \ell, [v_b, v_e)),$$

where r is the supporting inference record, δ is the selected action, π is the policy version, and ℓ is the loss-model version or documented operating point.

A provider may remain opaque with respect to data representation and numerical method, but not with respect to the statistical semantics of its output. Every score contribution declares whether it is a log likelihood, log-likelihood ratio, log potential, calibrated posterior, or an unsupported uncalibrated similarity. An uncalibrated similarity is not admitted directly into a Bayes-risk decision.

The durable ledger stores observable facts, explicit inferences, and explicit decisions. Probabilistic reasoning is delegated to the active workspace. The separation permits model replacement without changing the meaning of previously stored evidence.

4.5 Relations and invariants

A valid version satisfies the following invariants.

I1. **Type soundness.** Every endpoint conforms to the declared host or LI-ESKG relation domain and codomain.
I2. **Evidence immutability.** A committed observation payload is never overwritten.
I3. **At-most-one current decision.** An observation has at most one current assignment or rejection. It may have several historical decisions and may remain unresolved.
I4. **Provenance completeness.** Every current decision references an inference record, provider and model versions, calibration metadata, solver configuration, and policy version.
I5. **Canonical consistency.** Every active merge class has one deterministic representative.
I6. **Non-destructive revision.** Reassign, merge, split, promotion, correction, and revert append records and close validity intervals.
I7. **Atomic ledger visibility.** All ledger effects of a transaction become visible at one version, or none do.
I8. **Materialization traceability.** Every host write produced by LI-ESKG references the responsible decision and commit.
I9. **Idempotent replay.** Reapplying a committed idempotency key has no additional effect.

5 Dynamic Semantics

5.1 Three-state decomposition

The running system state is

$$\Xi_v = (\mathcal{K}_v, \mathcal{L}_v, \mathcal{W}_v),$$

where $\mathcal{K}_v$ is the authoritative host-graph view, $\mathcal{L}_v$ is the durable resolution state and ordered transaction log, and $\mathcal{W}_v$ is the active belief snapshot. A transition is a partial function

$$T : \Xi_v \times \mathcal{U} \to \Xi_{v+1},$$

where $\mathcal{U}$ is a validated command. Preconditions reject stale versions, type errors, unsupported host predicates, or invalid lifecycle changes.

Table 2: Core host and LI-ESKG relation roles. Every mutable LI-ESKG relation is versioned.

Relation	Domain	Codomain	Meaning
`Triggers`	State	Event	host state activates or enables an event
`LeadsTo`	Event	State	host event produces or leads to a resulting state
`Evolution`	State	State	host direct state evolution
`Contain`	Event	Event	host hierarchical or composite-event relation
`Occur`	Event	Physical node	host event-space attribution
`Influence`	Event	Physical node	host process or quality influence
`hasInference`	Observation	Inference	an inference record was produced for this evidence
`consideredCandidate`	Inference	Identity reference	the target belonged to the gated domain
`hasDecision`	Inference	Decision	a policy decision was produced from the inference
`selectedTarget`	Decision	Identity reference	the selected known or latent target
`promotedAs`	Latent identity	Host entity	the hypothesis was resolved and promoted
`mergedWith`	Latent identity	Latent identity	a versioned equivalence-generating decision
`supersedes/revokes`	versioned object	versioned object	a later record replaces or compensates an earlier interpretation

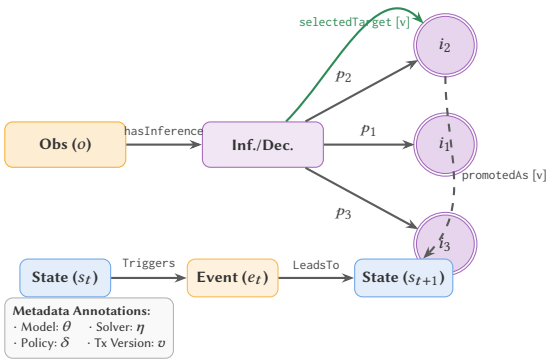

Figure 2: Typed LI-ESKG records beside the native ESKG relation pattern.

5.2 Observation transaction

For one observation, processing has five conceptual phases:

$$T_{\text{obs}} = T_{\text{materialize}} \circ T_{\text{commit}} \circ T_{\text{decide}} \circ T_{\text{infer}} \circ T_{\text{persist}}.$$

The first transition appends evidence or an immutable evidence reference. The second reads coherent host and provider snapshots, gates candidates, compiles the full batch factor graph, and computes a distribution. The third maps that distribution to an action. The fourth atomically appends the inference record, decision record, dependencies, and identity lifecycle events to the ledger. The fifth materializes an accepted host relation when the decision requires one. A version conflict restarts from the snapshot read. It does not reuse stale scores.

The ledger commit and an external Neo4j, Apache AGE, or other host write cannot generally be assumed to share one ACID transaction. A conforming implementation therefore uses a transactional outbox, idempotent host writes, and a materialization receipt. The host graph is unchanged until the materialization succeeds.

The allowed identity operations are *create*, *assign*, *abstain*, *reject*, *promote*, *reassign*, *revoke*, *merge*, *split*, *dormant*, *reactivate*, and *revert*. Merge is never the implicit side effect of a single assignment. It is a separate decision with separate evidence.

5.3 Promotion, merge, split, and reversal

A promotion maps a resolved latent identity i to an authoritative host entity p:

$$\text{Promote}_v(i, p).$$

After a successful promotion, i leaves the active workspace, while its evidence and decision lineage remain in the ledger. Later observations may still consider p through a lightweight host-entity reference and provider-owned summaries.

A split command supplies the intended partition and a dependency-aware reallocation plan. It identifies the merge decisions to close, observations to retain or reassign, summaries to rebuild, and downstream materializations to invalidate or mark ambiguous.

An undo never deletes the original command. A *strict revert* is rejected when active dependents exist. A *cascade revert* closes the transitive dependency closure. A *compensating revert* appends an explicit replacement when an operation is not mechanically invertible.

5.4 Current, historical, and bitemporal views

The contract distinguishes the current host and ledger view, a historical commit view, and an event-time view:

$$\text{CurrentView}(\Xi_v), \qquad \text{HistoricalView}(\Xi, v'), \qquad \text{AsOfEventTime}(\Xi, t_e).$$

A bitemporal query combines event time and commit version:

$$\text{View}(\Xi; t_e, v').$$

Queries must state whether they return only current materializations, all historically supported alternatives, or a distribution accompanied by its provenance and decision status.

6 Probabilistic Inference

6.1 Transient belief state

For each active latent identity or referenced host identity i, the hot belief state is

$$b_i = (i, \{u_{i,k}\}_{k \in K_i}, H_i, \ell_i, w_i),$$

where $u_{i,k}$ is a provider-owned opaque sufficient or bounded summary, H_i is a bounded recent history, ℓ_i is last activation time, and w_i is the applied commit version. The collection $\mathcal{B}_v = \{b_i\}$ is a cache and inference state. It is not the audit record.

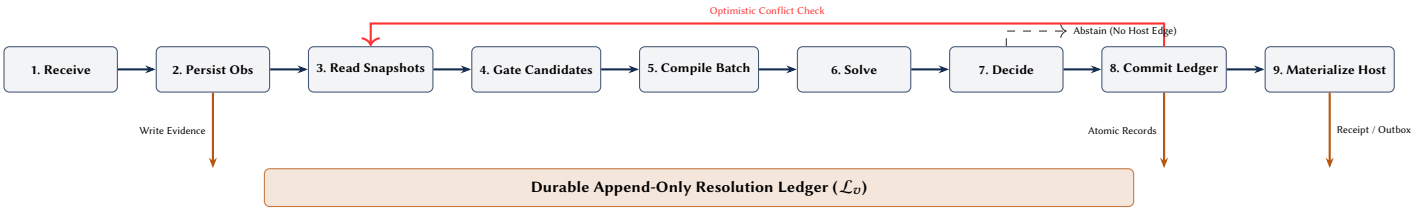

Figure 3: Versioned processing of one observation or one collective batch.

A proposed association does not update b_i. A committed association may update it. This rule limits self-reinforcing error. If soft updates are used, the implementation persists every weight and supports deterministic rollback. A split or reassignment rebuilds affected summaries from retained evidence or from a provider-defined reversible state; a bounded history is not assumed sufficient unless the provider declares it so.

6.2 Candidate generation

Inference over every known and latent identity is usually infeasible. Candidate generation applies a cascade:

$$\mathcal{I}_v^{\text{known}} \cup \mathcal{I}_v^{\text{latent}} \xrightarrow{\text{type/time gate}} C_1 \xrightarrow{\text{spatial gate}} C_2 \xrightarrow{\text{provider index}} C_v(o).$$

Cheap, high-recall filters precede expensive similarity evaluation. For ground-truth referent $i^*(o)$, candidate recall is

$$R_{\text{cand}} = \Pr[i^*(o) \in C_v(o)].$$

It upper-bounds assignment recall unless the system selects `new` and later promotes or merges the created identity. Candidate recall must therefore be reported separately from solver accuracy.

Candidate generation, profile construction, blocking, and modality-specific indexing are provider responsibilities. LI-ESKG validates provider versions, schema identifiers, content hashes, and declared score semantics.

6.3 Factor model

For a query batch $Q = \{o_1, \dots, o_n\}$, let $Z_j \in D_v(o_j)$ be the assignment variable. Auxiliary variables may represent motion, scene, relation, or capacity constraints. The compiler produces

$$F_v(Q) = (X, \Phi), \qquad p(x \mid Q, \mathcal{B}_v) = \frac{1}{Z_F} \prod_{\phi \in \Phi} \phi(x_{\text{scope}(\phi)}).$$

Every factor states its domain, inputs, provider and model versions, calibration procedure, and conditional-independence assumptions. Scores from correlated modalities must not be multiplied merely because they are available.

A provider contribution is typed as

$$c = (\alpha, s, \theta, \chi, \omega),$$

where α is the finite numeric value, s is the score semantics, θ is the model version, χ is the calibration artifact, and ω describes the validity domain. Admissible semantics include log likelihood, log-likelihood ratio, log potential, and calibrated posterior. A raw cosine similarity is not a probability and is rejected unless transformed under a documented model.

A spatial factor may use

$$\phi_{\text{sp}}(Z_j = i) = \mathcal{N}\left(x_j; \widehat{x}_{i,t_j}, \Sigma_{i,t_j} + R_j\right).$$

A biometric factor should transform distance into a calibrated likelihood ratio. The prior masses of `new` and `noise` must be explicit because they control identity proliferation and false attachment.

Numerical evaluation uses log potentials:

$$\log \widetilde{p}(x) = \sum_{\phi \in \Phi} \log \phi(x_{\text{scope}(\phi)}).$$

Normalization uses log-sum-exp. Embeddings may remain `f32`; accumulated log likelihoods should normally use `f64` unless benchmarks and error analysis justify otherwise.

6.4 Exactness and approximation

On an acyclic factor graph, Sum–Product computes exact marginals. On a cyclic graph, ordinary loopy Belief Propagation returns an approximation $\widehat{q}$, may oscillate, and may fail to converge. The solver records tolerance, iteration count, residual, stopping reason, and damping schedule.

Marginal inference and joint MAP inference are different objectives. Sum–Product estimates marginal distributions. Max–Product or another optimizer estimates

$$x^* = \arg\max_x p(x \mid Q, \mathcal{B}_v).$$

Independent maximization of marginals is not, in general, a joint MAP solution.

6.5 Exact local inference

Let $X_N \subset X$ be variables retained in a local graph and $X_{\bar{N}} = X \setminus X_N$. Eliminating external variables induces a boundary potential

$$\mu_{\partial N}(x_{\partial N}) = \sum_{x_{\bar{N}}} \prod_{\psi \in \Phi_{\bar{N}} \cup \Phi_\times} \psi(x_{\text{scope}(\psi)}),$$

where $\Phi_\times$ contains cross-boundary factors. Thus

$$p(x_N \mid Q, \mathcal{B}_v) \propto \mu_{\partial N}(x_{\partial N}) \prod_{\phi \in \Phi_N} \phi(x_{\text{scope}(\phi)}).$$

Theorem 1 (Exact localization). *Let $Z_q \in X_N$. Inference restricted to X_N preserves the global marginal of Z_q if the local solver includes the exact induced boundary potential $\mu_{\partial N}$. The result also holds when no factor crosses the boundary, or when the induced potential is constant with respect to every local variable that can influence Z_q.*

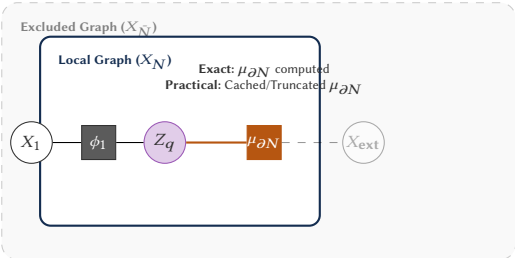

Figure 4: Local inference requires an induced boundary factor.

Proof. Partition the global factor product into internal, external, and cross-boundary terms. Marginalizing $X_{\bar{N}}$ produces exactly $\mu_{\partial N}$. Multiplying it by the internal factors gives the global unnormalized marginal on X_N. Normalization therefore yields the same marginal for Z_q. □

The presence of a Markov blanket does not alone establish exactness. External evidence may alter an unobserved blanket variable. Its induced message must be retained.

7 Decision Theory

Inference returns a distribution. A separate policy returns an action. Define

$$\mathcal{D}(o) = C_v(o) \cup \{\texttt{new}, \texttt{noise}, \texttt{abstain}\}.$$

Let $L(a, z)$ be the loss of action a when latent outcome $z \in D_v(o)$ is true. The Bayes action is

$$\delta(q) = \arg\min_{a \in \mathcal{D}(o)} \sum_{z \in D_v(o)} L(a, z) q(z).$$

Abstention has a declared review or delay cost. A false merge should usually cost more than temporary abstention because it contaminates several histories. Threshold rules are valid only when they implement an explicit loss model or a documented operating point.

Merge has its own decision rule. It may use posterior odds, constrained clustering, or human approval. The graph stores the supporting association records and policy version. A later split closes the merge interval and recomputes the current canonical map.

The one-step loss $L(a, z)$ is an explicit approximation when a false assignment or merge can contaminate future summaries and decisions. A deployment that models these downstream effects may replace the myopic policy with a sequential policy, but it must preserve the separation between inference records and decision records.

A decision that materializes a host ESKG relation identifies the host predicate role and target. For example, a resolved event-location hypothesis is materialized as $\text{Occur}(e, p)$, not as a generic LI-ESKG `refersTo` edge. LI-ESKG relations describe the resolution process; host relations describe accepted domain facts.

8 Interoperability Profiles

The numerical workspace is storage-format agnostic. It may use Rust structs, contiguous arrays, binary provider payloads, or remote services. RDF, PROV-O, and nanopublications define exchange and validation profiles rather than the inner-loop memory layout.

8.1 Host-schema profile

The motivating host profile uses the six relation roles defined in Section 3.1.1 of Zang et al. A deployment supplies a mapping

$$M_{\text{host}} : \mathcal{R}^{\text{ES}}_{\text{role}} \to \mathcal{R}_{\text{host}}$$

from Triggers, LeadsTo, Evolution, Contain, Occur, and Influence to the actual predicates used by the host graph. LI-ESKG neither invents an official `eskg:` namespace nor renames an existing host predicate.

8.2 RDF and PROV-O projection

A conforming implementation provides a deterministic projection

$$P_{\text{RDF}} : (\mathcal{K}_v, \mathcal{L}_v) \to \mathcal{D}_{\text{RDF}}.$$

The baseline projection is compatible with RDF 1.1. An RDF 1.2 profile may use triple terms and reifiers to describe candidate and disputed propositions without asserting them as current host facts. Observation references, inference runs, decision activities, model artifacts, host snapshots, revisions, invalidations, and responsible agents are aligned with PROV-O classes and properties.

The internal ledger need not store RDF triples. It must, however, retain enough information to generate the projection without inventing missing provenance after the fact.

8.3 Nanopublication and SHACL profiles

A durable decision or materialized assertion may be exported as one or more nanopublications. A nanopublication is an immutable exchange unit containing an assertion graph, assertion provenance, and publication information. It is not an LI-ESKG commit and does not replace atomic commit, idempotency, conflict detection, or dependency-aware reversal.

The RDF projection is accompanied by SHACL shapes that test cardinalities, score semantics, model and policy versions, host-snapshot references, decision uniqueness, promotion targets, and revocation links. Conformance is therefore a testable property of the export profile rather than a claim of "perfect interoperability."

9 Formal Properties

9.1 Host non-interference before materialization

Proposition 1 (Host isolation). *Let u be a valid LI-ESKG command whose ledger commit has not produced a successful host materialization receipt. Then*

$$\mathcal{K}_{v+1} = \mathcal{K}_v.$$

Proof. Candidate generation, factor compilation, inference, decision, and ledger persistence modify only $\mathcal{W}_v$ or $\mathcal{L}_v$. By contract, host writes occur only through the materialization adapter after a committed decision. Without a successful materialization receipt, no host transition has occurred. □

Corollary 1 (Native host semantics). *A successful materialization changes the host graph only through predicates declared by the host-schema profile. LI-ESKG does not replace* Triggers, LeadsTo, Evolution, Contain, Occur, *or* Influence *with generic identity predicates.*

9.2 Invariant preservation

Theorem 2 (Preservation under valid commands). *Let* $\mathrm{Valid}(\Xi_v)$ *denote invariants I1–I9. For every command* u *in* create, assign, abstain, reject, promote, reassign, merge, split, retire, reactivate, *and* revert,

$$\mathrm{Valid}(\Xi_v) \wedge \mathrm{Pre}_u(\Xi_v) \Longrightarrow \mathrm{Valid}(T_u(\Xi_v)).$$

Proof. Each command is admitted only after type, version, lifecycle, dependency, and idempotency checks. The transition appends new records and closes validity intervals rather than overwriting evidence. The commit writes all ledger effects atomically. Materialization is idempotent and records its decision and commit. Merge and split recompute the active canonical map; split additionally closes or invalidates every declared dependent object. These command-specific postconditions establish I1–I9. □

9.3 Version reconstruction

Theorem 3 (Replay). *Assume an append-only, totally ordered resolution ledger; deterministic transitions; a valid checkpoint at version* s*; and non-destructive revisions. Then every committed ledger and workspace view* $(\mathcal{L}_v, \mathcal{W}_v)$*,* $v \geq s$*, can be reconstructed from the checkpoint and the log segment* $(s, v]$*. Host materializations can be replayed from committed outbox entries and verified against their receipts.*

Proof. Apply the ordered transition sequence after the checkpoint. Determinism fixes every next state. Idempotency prevents duplicate effects. Non-destructive revision ensures that no later operation removes input required to reconstruct an earlier view. Induction on the log prefix gives the result. Host replay follows from idempotent materialization keys and persisted write plans. □

9.4 Resource bounds

Let M be the number of active known or latent identity references, C the maximum number of provider summaries per identity, d the maximum summary size, and K the retained recent-history bound.

Proposition 2 (Hot belief memory). *Excluding candidate indexes, temporary factors, allocator overhead, checkpoints, the durable ledger, and the host graph, the hot belief state requires*

$$O(M(Cd + K))$$

memory.

Proof. Each active identity reference stores at most C summaries of size at most d and K bounded history entries. Sum over M references. □

Persistent storage is not history-independent. Retaining every observation, inference record, decision record, dependency, and materialization receipt costs at least

$$\Omega(|\mathcal{O}| + |\mathcal{F}| + |\mathcal{D}| + |\mathcal{L}|).$$

Host-graph storage is additional and backend-dependent.

Let T be the number of full message-passing iterations. A generic factor-to-variable update has worst-case cost proportional to each recipient-domain size times the product of the remaining scoped domain sizes. Hence loopy Belief Propagation costs

$$O\left(T \sum_{\phi \in \Phi} \sum_{Z \in \mathrm{scope}(\phi)} \prod_{Y \in \mathrm{scope}(\phi) \setminus \{Z\}} |D(Y)|\right).$$

Candidate size controls $|D(Z)|$. Factor arity controls the product. The history bound K alone does not bound inference latency.

If a checkpoint at sequence s has size $|L_s|$ and Δ_s later log records are replayed, sequential recovery costs

$$O(|L_s| + \Delta_s)$$

I/O and transition work, excluding external model loading, host verification, and index rebuilding.

10 Rust Reference Design

The implementation should make invalid states difficult to express. It should also preserve locality in hot loops. These goals favor closed enums, typed keys, and modality-specific columns over `Box<dyn Any>` and nested trait objects.

10.1 Typed identifiers and storage

The implementation should make invalid states difficult to express. Typed keys prevent accidental cross-indexing. Stable ledger identifiers are distinct from host entity identifiers and from provider-owned opaque keys.

Listing 1: Typed keys, host references, and opaque provider handles.

```rust
use bytes::Bytes;
use slotmap::{new_key_type, DenseSlotMap, SlotMap};
use std::sync::Arc;

new_key_type! {
    pub struct ObservationId;
    pub struct LatentIdentityId;
    pub struct InferenceId;
    pub struct DecisionId;
}

#[derive(Clone, Copy, Debug, Eq, PartialEq, Hash)]
pub struct ProviderId(pub u64);
#[derive(Clone, Copy, Debug, Eq, PartialEq, Hash)]
pub struct SchemaId(pub u64);

#[derive(Clone, Debug)]
pub struct HostEntityRef {
    pub backend: u32,
    pub key: Arc<str>,
    pub snapshot_version: u64,
}

#[derive(Clone, Debug)]
pub struct SummaryHandle {
    pub provider: ProviderId,
    pub schema: SchemaId,
    pub opaque_key: Bytes,
    pub version: u64,
    pub content_hash: [u8; 32],
}

pub struct Stores {
    pub observations: SlotMap<ObservationId, Observation>,
    pub latent: DenseSlotMap<LatentIdentityId, IdentityMeta>,
    pub inferences: SlotMap<InferenceId, InferenceRecord>,
    pub decisions: SlotMap<DecisionId, DecisionRecord>,
}
```

Large raw media should live in an object store or blob segment. The ledger stores a validated reference and content hash. `Bytes` is suitable for cheaply cloned, sliceable binary buffers. `Arc<[f32]>`

remains suitable inside a provider adapter for immutable shared embeddings. Neither choice makes copying free; both remove avoidable deep copies.

10.2 Opaque provider dispatch

The original closed `SummaryRef` enum makes the core aware of four concrete modalities and their Rust types. The revised boundary keeps provider payloads opaque while requiring typed statistical outputs. Dynamic dispatch occurs once per provider or batch, not once per candidate inner-loop operation.

Listing 2: Provider-agnostic candidate and factor interface.

```rust
#[derive(Clone, Copy, Debug, Eq, PartialEq)]
pub enum ScoreSemantics {
    LogLikelihood,
    LogLikelihoodRatio,
    LogPotential,
    CalibratedPosterior,
}

pub struct ScoreContribution {
    pub value: f64,
    pub semantics: ScoreSemantics,
    pub provider: ProviderId,
    pub model_version: u64,
    pub calibration_id: u64,
}

pub trait FactorProvider: Send + Sync {
    fn provider_id(&self) -> ProviderId;

    fn generate_candidates(
        &self,
        batch: &[ObservationEnvelope],
        snapshot: &HostSnapshot,
        out: &mut CandidateBuffer,
    ) -> Result<(), ProviderError>;

    fn emit_factors(
        &self,
        batch: &[ObservationEnvelope],
        candidates: &CandidateBuffer,
        context: &FactorContext,
        out: &mut FactorBuffer,
    ) -> Result<(), ProviderError>;
}
```

The core does not inspect the internal structure of a face centroid, Splink profile, Kalman state, neural embedding, relational profile, or remote proprietary service. It does inspect schema identifiers, versions, hashes, finite numeric values, and score semantics. Provider agnosticism therefore concerns implementation, modality, and payload representation; it does not mean statistical indifference.

10.3 Candidate buffers and allocation

Candidate lists are usually short after blocking. `SmallVec<[IdentityId; N]>` stores up to N keys inline and spills to the heap when needed. The value of N must be chosen from observed percentiles. A large inline capacity bloats every request and harms cache density.

Each worker should own reusable scratch storage:

- a candidate buffer with reserved capacity;
- contiguous score and message arrays;
- factor adjacency offsets;
- a heap or partial-selection buffer for top-k;
- temporary linear-algebra workspace.

Clear these buffers without releasing capacity between requests. Avoid a `HashMap` in a dense inner loop. Map stable candidate keys to batch-local integer offsets once, then use slices. Use `hashbrown` for auxiliary indexes when hash lookup is justified. Reserve capacity before batch insertion. Retain a collision-resistant hasher for attacker-controlled keys.

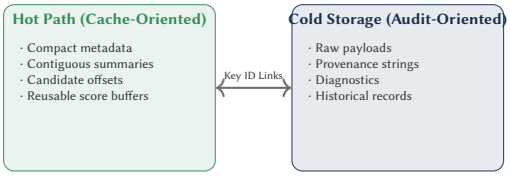

Figure 5: Hot–cold separation for the Rust implementation.

For large sparse support sets, `roaring::RoaringBitmap` can reduce memory and accelerate set operations. It is not automatically superior for small sets or dense contiguous ranges. Benchmark the actual cardinality distribution.

10.4 Data layout

The primary hot path scores many candidates against one observation. A structure-of-arrays layout is often preferable:

`means[]`, `precisions[]`, `last_seen[]`, `quality[]`.

It reduces unrelated cache-line loads and exposes contiguous numeric slices to optimized kernels. Provenance strings, raw payloads, diagnostics, and long histories are cold data. Store them separately and access them after the decision.

Use compact integer codes for modality, source, and lifecycle status. Intern repeated labels. Store event time as a fixed-width integer when the domain permits it. Keep embeddings in `f32`. Use `f64` for sensitive reductions. Measure quantization separately before using lower precision.

10.5 Parallelism and backpressure

Candidate scoring is CPU-bound. `rayon` is appropriate for batches large enough to amortize scheduling. Parallelize over independent observations or large candidate chunks. Do not spawn one task per factor. Use one configured pool, avoid nested oversubscription, and preserve a deterministic reduction order where reproducibility matters.

The ingestion path should use bounded `crossbeam-channel` queues. Bounded queues impose backpressure and cap in-flight memory. An unbounded queue converts overload into memory growth. I/O runtimes may orchestrate network and storage calls, but CPU scoring should remain on a dedicated pool.

Read-mostly model parameters and candidate-index snapshots can be published through `arc-swap`. A request reads one immutable snapshot and uses it throughout inference. Publication then replaces the snapshot atomically. This avoids holding a global read lock across a query. It does not remove the need for version checks at commit.

10.6 Linear algebra, retrieval, and persistence

Use `nalgebra` for low-dimensional fixed-size vectors and matrices. Its statically sized objects avoid heap allocation for small dimensions. Use `faer` for medium or large dense matrices and decompositions. Do not include both libraries in the same hot operation

without a measured reason. Repeated data conversion may dominate the intended gain.

Approximate nearest-neighbor retrieval may use an HNSW implementation such as `instant-distance`. It is a candidate generator, not an inference engine. Measure recall against exact search. Persist index configuration and snapshot version. A stale index may return false candidates or omit the correct identity.

An embedded deployment may use `rocksdb` for the durable resolution ledger, ACID transactions, MVCC reads, and copy-on-write B-tree persistence. Group related ledger writes into one transaction. Batch inference and decision records when the latency budget permits. Keep large blobs outside the key-value store. Neo4j, Apache AGE, RDF stores, or other property-graph systems remain authoritative host backends behind an idempotent materialization adapter. For distributed deployments, the ledger backend must still implement the same snapshot, idempotency, and atomic-commit contract.

10.7 Transactional pipeline

Listing 3: Allocation-aware collective batch skeleton.

```rust
pub fn process_batch(
    batch: &[ObservationId],
    ctx: &Context,
    scratch: &mut WorkerScratch,
) -> Result<Vec<Commit>, PipelineError> {
    let host = ctx.host_snapshots.load_full();
    let providers = ctx.providers.load_full();

    scratch.reset_batch();
    ctx.load_observations(batch, &mut scratch.observations)?;

    for provider in providers.iter() {
        provider.generate_candidates(
            &scratch.observations,
            &host,
            &mut scratch.candidates,
        )?;
        provider.emit_factors(
            &scratch.observations,
            &scratch.candidates,
            &scratch.factor_context,
            &mut scratch.factors,
        )?;
    }

    let graph = ctx.compiler.compile_batch(
        &scratch.observations,
        &scratch.candidates,
        &scratch.factors,
        &mut scratch.graph,
    )?;
    let posterior = ctx.solver.solve_graph(
        graph,
        &mut scratch.messages,
    )?;
    let decisions = ctx.policy.decide_batch(&posterior)?;

    let commits = ctx.ledger.commit_batch(
        host.version,
        &scratch.observations,
        &posterior,
        &decisions,
    )?;
    ctx.materializer.enqueue(&commits)?;
    Ok(commits)
}
```

The code deliberately exposes reusable buffers, coherent host and provider snapshots, and one collective factor graph for the batch. Production code should stream commits or reuse an output buffer if returning a new vector becomes material. A version conflict invalidates the whole decision that depended on the stale snapshot. Every commit carries the provider, model, calibration, solver, policy, host-snapshot, and idempotency metadata required by the provenance invariant.

10.8 Recommended crates

Table 3 lists focused components. Versions should be pinned in `Cargo.lock`, audited, and updated through reproducible benchmarks.

10.9 Performance method

Optimization must follow measurement. Instrument candidate count, gate time, scoring time, solver iterations, commit time, allocations, queue depth, and snapshot version. Report median and tail latency. Use `criterion` for kernels. Use end-to-end replayable traces for pipeline tests. Inspect profiles and hardware counters before changing layout.

The highest-leverage sequence is usually:

(1) increase cheap-gate recall while reducing expensive candidate count;
(2) remove allocation and cloning from the query loop;
(3) make numeric arrays contiguous and precompute invariant terms;
(4) batch scoring and persistence;
(5) parallelize only after the sequential hot path is compact;
(6) quantify the accuracy cost of ANN, truncation, and reduced precision.

Compiler settings are deployment-specific. Benchmark whole-program optimization, a single code-generation unit, and a native CPU target only on the intended deployment hardware. Keep a portable build. Do not treat build flags as an architectural substitute for better candidate gating and data layout.

11 Evaluation Protocol

The architecture yields falsifiable questions.

RQ1: Resolution.
Does collective multimodal inference improve assignments over independent pairwise matching?

RQ2: Calibration.
Do reported probabilities match empirical frequencies?

RQ3: Revision.
Does the system recover from false assignments and false merges without losing history?

RQ4: Locality.
What accuracy and latency are lost when boundary factors are cached, truncated, or omitted?

RQ5: Scale.
How do throughput, tail latency, memory, and storage vary with active identities, candidate count, batch size, factor arity, and iterations?

RQ6: Recovery.
Does replay reconstruct exactly the same versioned ledger and host materialization set after injected crashes?

RQ7: Provider agnosticism.
Do providers producing the same canonical factors yield the same decision projection, while retaining distinct provenance?

Table 3: Recommended Rust crates and decision criteria.

Crate	Role	Use when	Main caution
`slotmap`	typed stable keys; dense and secondary maps	in-memory graph objects need safe stable references	choose dense versus random-access layout from profiles
`smallvec`	inline short vectors	candidate counts are usually below a measured percentile	excessive inline capacity increases every request size
`bytes`	shared, sliceable byte buffers	encoded payloads cross pipeline stages	reference counting is not free; avoid for tiny copied values
`hashbrown`	high-performance hash indexes	sparse auxiliary lookup is unavoidable	reserve capacity; secure untrusted-key hashing
`roaring`	compressed integer sets	large sparse memberships require union/intersection	benchmark against sorted vectors and bitsets
`rayon`	data-parallel CPU work	scoring batches amortize scheduling	cap pool size; avoid nested parallelism
`crossbeam-channel`	bounded work queues	synchronous stages need backpressure	size queues from latency and memory budgets
`arc-swap`	read-mostly immutable snapshots	models or indexes are read often and replaced rarely	a request must keep one coherent snapshot
`nalgebra`	small fixed-size linear algebra	geometry and covariance dimensions are low	dynamic large matrices are not its primary strength
`faer`	medium/large dense algebra	factor compilation or updates use larger matrices	conversion and parallel thresholds must be measured
`rocksdb`	embedded transactional persistence	a single-node ACID store is sufficient	separate large blobs; batch durable writes
`tracing`	structured spans and events	latency must be attributed by stage and version	sample high-volume events
`criterion`	statistical microbenchmarks	kernels and allocation changes need regression tests	complement with end-to-end load tests

RQ8: Interoperability.
Do RDF/PROV-O exports satisfy the published SHACL shapes and round-trip through supported host adapters?

Evaluation should combine synthetic and real data. A synthetic generator should control identity count, motion, sensor noise, missing modalities, correlated errors, clutter, arrival, dormancy, reappearance, and ground-truth merge or split events. A real benchmark should contain timestamps, repeated observations per entity, and at least two modalities or relational contexts. Ground truth must remain inaccessible to candidate generation and inference.

Required baselines include independent pairwise matching, a calibrated Fellegi–Sunter model, a Bayesian or graphical entity-resolution model, an incremental linkage method, a tracking baseline when trajectories exist, LI-ESKG without collective factors, and an oracle candidate generator. Provider-side systems such as Splink or domain-specific resolvers are evaluated as interchangeable candidate-and-factor producers rather than as functionality reimplemented by the LI-ESKG core.

Resolution metrics include pairwise precision, recall, and F_1; B-cubed scores; adjusted Rand index; false-merge rate; false-split rate; and new-identity precision. Calibration metrics include negative log likelihood, Brier score, expected calibration error, reliability diagrams, and risk–coverage curves. Systems metrics include throughput, median latency, p_{95}, p_{99}, peak resident memory, bytes allocated per observation, queue depth, checkpoint pause, recovery time, write amplification, and storage growth.

Ablations remove each provider factor, collective factors, boundary messages, abstention, ANN retrieval, reversible merge support, and provenance export. Fault injection stops the process before and after each ledger write and host materialization. Recovery is correct only if the reconstructed ledger hash, decision history, dependency closure, and authoritative host materialization set equal the failure-free execution at the same commit version.

12 Applications

LI-ESKG is pertinent when identity is uncertain, evidence arrives continuously, and event-state history must remain auditable.

Multi-sensor tracking. Face, voice, location, and text observations may support the same person hypothesis. The system retains each item of evidence and each revision.

Robotics. A robot may lose and reacquire a person, vehicle, or object. Dormancy distinguishes temporary absence from deletion.

Industrial digital twins. Sensors and business systems may use inconsistent identifiers for one physical part. Versioned associations expose how its history was assembled.

Cybersecurity. Accounts, devices, addresses, and sessions may be attributed to a latent actor. High-cost false merges motivate abstention and review.

Document and GraphRAG pipelines. Extracted mentions can remain probabilistic while events and states are indexed. Retrieval can select a current canonical view or a historical version.

The architecture is unnecessary when every entity has a reliable identifier, the data are static, or one-time deduplication is sufficient.

13 Limitations and Threats to Validity

LI-ESKG does not solve entity resolution. Accuracy depends on gating, factors, training data, calibration, and the loss function. Misspecification can produce confident errors. Candidate retrieval may

exclude the correct identity. Correlated modalities may invalidate a factorization.

The architecture guarantees internal consistency, not one identity per physical entity. Exact inference remains limited by graph structure. Exact boundary factors may cost as much as the global computation they summarize. Loopy Belief Propagation remains approximate.

The minimal domain assumes that each observation refers to at most one physical entity, although inference may be collective over a batch. Joint creation of several anonymous identities in one batch requires a partition model or explicit anonymous variables. Long-term re-identification may dominate error and latency.

The proposed Rust choices and host-adapter patterns are design recommendations, not benchmark results. A crate can regress, an access distribution can change, and a faster microkernel can increase end-to-end latency through allocation or contention. Dependency versions, features, target hardware, and datasets must be reported.

Privacy, retention, access control, and deletion are deployment requirements. Logical immutability does not override legal deletion. A compliant design may retain a non-identifying audit tombstone while deleting protected payloads.

Finally, event-state transitions encode operational and temporal dependence. They do not establish interventionist causality. Causal claims require additional assumptions and methods.

14 Conclusion

LI-ESKG places observations and identities at different semantic levels. Observations are immutable evidence. Identities are versioned explanatory hypotheses. Inference distributions and decisions remain persistent in the resolution ledger. Decisions may abstain. Merges are non-destructive. Splits, reassignments, promotions, and reversals preserve history.

The authoritative knowledge graph stores accepted domain facts and the native ESKG relations declared by its host profile. Probabilistic reasoning is delegated to an ephemeral inference layer, while a durable ledger stores evidence, alternatives, decisions, dependencies, and materialization receipts. The architecture accepts any provider and solver that return typed statistical contributions or distributions with diagnostics. It states exactness only under valid graphical conditions. It exposes the variables that govern memory and runtime.

LI-ESKG is potentially new as a formalization and integration architecture, but not as a probabilistic method or an entity-resolution algorithm. Its value lies in the contract between external resolution providers, collective inference, the temporary latent-identity workspace, durable revision, host-graph materialization, interoperability, and audit. Its practical value remains an empirical question. The protocol in this paper defines how to answer it.

References

[1] Bilal Abu-Salih. 2021. Domain-specific Knowledge Graphs: A Survey. *Journal of Network and Computer Applications* 192 (2021), 103076. doi:10.1016/j.jnca.2021.103076

[2] Stephen H. Bach, Matthias Broecheler, Bert Huang, and Lise Getoor. 2017. Hinge-Loss Markov Random Fields and Probabilistic Soft Logic. *Journal of Machine Learning Research* 18, 109 (2017), 1–67. http://jmlr.org/papers/v18/15-631.html

[3] Indrajit Bhattacharya and Lise Getoor. 2007. Collective entity resolution in relational data. *ACM Trans. Knowl. Discov. Data* 1, 1 (March 2007), 5–es. doi:10.1145/1217299.1217304

[4] Peter Christen. 2012. *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection.* Springer, New York, NY.

[5] Ivan Fellegi and Alan Sunter. 1969. A Theory for Record Linkage. *J. Amer. Statist. Assoc.* 64, 328 (1969), 1183–1210. doi:10.1080/01621459.1969.10501049

[6] Paul Groth, Andrew Gibson, and Jan Velterop. 2010. The Anatomy of a Nanopublication. *Information Services and Use* 30, 1–2 (2010), 51–56. doi:10.3233/ISU-2010-0613

[7] Daphne Koller and Nir Friedman. 2009. *Probabilistic Graphical Models: Principles and Techniques.* MIT Press.

[8] Frank R. Kschischang, Brendan J. Frey, and Hans-Andrea Loeliger. 2001. Factor Graphs and the Sum-Product Algorithm. In *Proceedings of the IEEE.* Vol. 47. 498–519. doi:10.1002/0471748602.ch6

[9] Tobias Kuhn, Christine Chichester, Michael Krauthammer, Núria Queralt-Rosinach, Ruben Verborgh, Georgios Giannakopoulos, Axel-Cyrille Ngonga Ngomo, Raffaele Viglianti, and Michel Dumontier. 2016. Decentralized Provenance-Aware Publishing with Nanopublications. *PeerJ Computer Science* 2 (2016), e78. doi:10.7717/peerj-cs.78

[10] Mauricio Sadinle. 2014. Detecting duplicates in a homicide registry using a Bayesian partitioning approach. *The Annals of Applied Statistics* 8, 4 (2014), 2304–2334. doi:10.1214/14-AOAS772

[11] Rebecca C. Steorts, Rob Hall, and Stephen E. Fienberg. 2015. A Bayesian Approach to Graphical Record Linkage and De-duplication. arXiv:1312.4645 [stat.ME] https://arxiv.org/abs/1312.4645

[12] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. 2005. *Probabilistic Robotics.* MIT Press.

[13] Yunong Tian, Ning Wang, and Anshun Zhou. 2026. CrossER: A robust and adaptable generalized entity resolution framework for diverse and heterogeneous datasets. *Information Systems* 135 (2026), 102609. doi:10.1016/j.is.2025.102609

[14] W3C. [n. d.]. RDF 1.2 Concepts and Abstract Syntax. W3C Specification. https://www.w3.org/TR/rdf12-concepts/

[15] W3C. 2013. PROV-O: The PROV Ontology. W3C Recommendation. https://www.w3.org/TR/prov-o/

[16] W3C. 2017. Shapes Constraint Language (SHACL). W3C Recommendation. https://www.w3.org/TR/shacl/

[17] Jonathan S. Yedidia, William T. Freeman, and Yair Weiss. 2003. Understanding Belief Propagation and Its Generalizations. *Exploring Artificial Intelligence in the New Millennium* 8 (2003), 239–269.

[18] T. Zang, Y. Li, M. Yang, and P. Jiang. 2026. Event-state knowledge graph-enhanced GraphRAG for order-driven 3D printing supply chain configuration. *Expert Systems with Applications* 317 (2026), 121401–121415. doi:10.1016/j.eswa.2025.121401